# Tool Mapping Protocol

## A deterministic mapping layer for tools, actions, APIs, queries, workflows, and agents

TMP Project

2026-06-03

## Contents

# 1 Executive Summary

Tool Mapping Protocol, or TMP, is a proposed protocol for mapping human or agent intent to verified executable surfaces. In the first implementation, TMP appears as a Rust CLI that maps natural-language requests to command-line schemas. That is useful, but it is not the full design.

The broader TMP design is a general mapping layer for:

- CLI commands.
- HTTP and local API endpoints.
- SQL queries and database operations.
- Workflow actions and multi-step automations.
- Custom scripts.
- Terminal autocomplete and tab completion.

- AI agent tool use.

TMP is not only a way to run commands. It is a way to describe what actions exist, what parameters they accept, where parameter values come from, what side effects they may have, how they are verified, and how a user or agent can safely invoke them.

The core claim is:

> Agents and terminals should not guess executable behavior. They should resolve intent through a verified map.

This creates three practical benefits:

- Grounded results: the output is tied to a known operation, not a hallucinated command or API call.
- Lower tool-call usage: an agent can use one TMP resolution call instead of repeatedly probing files, help pages, schemas, scripts, and APIs.
- Lower token usage: compact mappings replace repeated long-context inspection.

TMP can support both interactive humans and automated agents. A terminal can use TMP for completions. An agent can use TMP for grounded action selection. A registry can distribute verified maps. A schema generator can help create maps. A benchmark harness can measure whether these claims hold.

## 2 The Bigger Idea

Most developer tools expose useful operations, but they expose them in inconsistent ways. A CLI has flags. An API has endpoints. SQL has tables and query constraints. A workflow has steps. A script has inputs. A terminal completion engine has candidate values. An AI agent has tools with JSON schemas.

All of these are different forms of the same underlying problem:

```
intent + context -> valid operation + filled parameters + known effect
```

TMP treats that as a first-class protocol problem.

Instead of building a separate resolver for every surface, TMP defines a common map. The map says:

- What surface exists.
- What operations exist on that surface.
- What parameters each operation accepts.
- Which parameters are required.
- How valid parameter values can be discovered.
- What context is needed.
- What effect or risk the operation carries.
- How the operation was verified.

- How the operation should be invoked.

This lets different frontends use the same mapping:

- A terminal asks for tab-completion candidates.
- An AI agent asks to resolve "run unit tests".
- A workflow engine asks which action can deploy staging.
- A database assistant asks which SQL queries are allowed.
- A script runner asks what script arguments are valid.

The protocol is the shared layer. The current `tmp` CLI is one adapter and one developer workflow around that layer.

## 3 Why This Matters Now

Coding agents are becoming normal development tools. They can inspect code, make changes, run tests, and call tools. But tool use is still inefficient and error-prone.

Without a mapping layer, an agent often has to perform many exploratory calls:

1. List files.
2. Read package files.
3. Read README files.
4. Search scripts.
5. Run help commands.
6. Inspect shell history or docs.
7. Guess the final command.
8. Run it.
9. Recover if it was wrong.

This creates waste:

- More tool calls.
- More tokens.
- More latency.
- More opportunities for hallucinated flags or wrong assumptions.
- More user interruptions.

TMP compresses that flow:

1. Compile or load a verified map.
2. Resolve intent against the map.
3. Invoke the mapped operation.

When the map is missing or uncertain, TMP should say so. That failure is valuable because it prevents a guessed action from being presented as grounded.

# 4 Use Cases

## 4.1 Terminal Autocomplete

Terminals such as Warp, shell plugins, or custom developer terminals can use TMP to provide context-aware completion.

Example:

```
git checkout <TAB>
```

TMP can map `<branch>` to `git:branches` and return current branch names.

Example:

```
cargo run --bin <TAB>
```

TMP can map `<bin>` to `cargo:bins` and return binaries detected in the current workspace.

This is not only convenience. It makes completion project-aware and schema-aware. The terminal does not need to scrape every help page repeatedly. It can ask TMP for the operation map and dynamic parameter values.

## 4.2 AI Agent Tool Use

An AI agent can use TMP as a grounding tool.

Example:

```
User: Run the parser unit tests.
Agent: tmp resolve "run parser unit tests"
TMP: cargo test parser
Agent: tmp run
```

The agent does not need to infer the command from scratch. It asks TMP for the mapped operation. If TMP cannot resolve the request, the agent receives a visible failure and can inspect the repo, ask the user, or generate a missing schema.

## 4.3 API Action Mapping

TMP can map API operations.

Example:

```
intent: "create a staging deployment"
surface: api
operation: POST /deployments
parameters:
  environment: staging
  ref: current git branch
effect: deployment
risk: high
approval: required
```

The protocol can describe the endpoint, method, required fields, authentication context, valid values, response schema, and risk level. A terminal, workflow runner, or agent can all use the same map.

## 4.4 SQL Query Mapping

TMP can map database operations without allowing arbitrary SQL generation.

Example:

```
intent: "show recent failed jobs"
surface: sql
operation: recent_failed_jobs
query_template: SELECT id, status, created_at FROM jobs WHERE status = ? ORDER BY
created_at DESC LIMIT ?
parameters:
  status: failed
  limit: 50
effect: read-only
risk: low
```

This matters because free-form SQL generation can be dangerous. TMP can define allowed query templates, parameter sources, read/write classification, and database context.

## 4.5 Workflow and Action Mapping

TMP can map multi-step workflows.

Example:

```
operation: release_candidate
steps:
  - run tests
  - build package
  - generate changelog
  - publish draft artifact
effect: write-local + network
risk: high
approval: required
```

The map can expose a workflow as a single operation while preserving its internal steps and risk metadata.

## 4.6 Custom Script Mapping

Many teams rely on local scripts that are not documented well.

TMP can map:

- Script path.
- Arguments.
- Environment variables.
- Working directory.

- Generated files.
- Exit behavior.
- Dynamic completions.
- Risk level.

This turns ad hoc scripts into inspectable operations.

# 5 Core Concept

TMP should be understood as a mapping protocol, not only a command schema.

The fundamental unit is an operation.

```
surface -> operation -> parameters -> resolvers -> invocation -> effect
```

## 5.1 Surface

A surface is where the operation lives.

Examples:

- CLI
- API
- SQL
- Workflow
- Script
- Filesystem
- Agent tool

The current implementation focuses mostly on CLI surfaces. The grand design should support multiple surfaces through adapters.

## 5.2 Operation

An operation is something that can be invoked.

Examples:

- `cargo test`
- `POST /deployments`
- `recent_failed_jobs`
- `release_candidate`
- `scripts/sync-data.sh`

Each operation should have a stable identity, description, invocation template, parameters, effect metadata, and verification status.

## 5.3 Parameter

A parameter is an input required by an operation.

Examples:

- Cargo package.
- Git branch.
- API environment.
- SQL status filter.
- Workflow target.
- Script path.

Parameters can be required or optional. They can have types, defaults, allowed values, and dynamic resolvers.

### 5.4 Resolver

A resolver discovers valid parameter values.

Examples:

- `git:branches`
- `cargo:bins`
- `npm:scripts`
- `api:environments`
- `sql:tables`
- `workflow:targets`
- `script:args`

Resolvers are one of TMP's most important ideas. They make maps dynamic without making the agent guess.

### 5.5 Context

Context is the current state needed to resolve operations.

Examples:

- Repository root.
- Current branch.
- Build system.
- Active database.
- API base URL.
- Environment.
- User permissions.
- Known generated files.

TMP should compile context into compact artifacts for terminals and agents.

### 5.6 Effect

An effect describes what can happen when an operation runs.

Examples:

- Read-only.

- Local write.
- Build/test.
- Network.
- Database write.
- Deployment.
- Destructive.

Effects let clients decide when to ask for approval.

### 5.7 Evidence

Evidence explains why the map should be trusted.

Examples:

- Parsed from help text.
- Verified by a human.
- Tested against command output.
- Imported from a trusted registry.
- Generated by an agent and reviewed.
- Signed by a publisher.

Evidence is what separates a draft map from a trusted operation.

## 6 Protocol Shape

A future TMP schema should evolve from CLI-specific command records into a generalized operation map.

Illustrative shape:

```
{
  "meta": {
    "tool": "example",
    "version": 1,
    "verified": false
  },
  "surfaces": [
    {
      "kind": "cli",
      "name": "cargo",
      "operations": [
        {
          "id": "cargo.test",
          "intent": ["test", "run tests", "unit tests"],
          "template": "cargo test <test_filter>",
          "parameters": [
            {
              "name": "test_filter",
              "type": "string",
              "required": false,
```

```
            "resolver": "cargo:tests"
          }
        ],
        "effect": "build-test",
        "risk": "low",
        "verified": true
      }
    ]
  }
 ]
}
```

The current Rust schema can remain a CLI-oriented subset while the broader protocol is designed.

# 7 Sequential Architecture

TMP should be easy to explain as a sequence.

## 7.1 Stage 1: Discover

Collect raw surface information:

- CLI help text.
- OpenAPI specs.
- SQL schema.
- Workflow definitions.
- Script signatures.
- Existing docs.
- Repository files.

## 7.2 Stage 2: Map

Convert raw information into operation maps:

- Operations.
- Parameters.
- Resolvers.
- Effects.
- Context requirements.

This can be deterministic, agent-assisted, or registry-imported.

## 7.3 Stage 3: Verify

Prove the map is useful:

- Run help checks.
- Test parameter resolution.
- Execute safe dry runs.
- Validate SQL templates.
- Confirm API schemas.

- Review effect labels.

### 7.4 Stage 4: Compile

Compile the active map for the current context:

- Filter irrelevant operations.
- Resolve dynamic values.
- Emit terminal completion data.
- Emit agent-readable context.
- Emit machine-readable operation maps.

### 7.5 Stage 5: Resolve

Resolve intent:

```
"run parser tests" -> operation: cargo.test -> command: cargo test parser
```

or:

```
"show failed jobs" -> operation: sql.recent_failed_jobs -> query template +
parameters
```

### 7.6 Stage 6: Invoke

Invoke the mapped operation:

- Shell command.
- API request.
- SQL query.
- Workflow runner.
- Script process.

Approval can be required based on effect and risk metadata.

### 7.7 Stage 7: Measure

Record whether TMP helped:

- Did it resolve correctly?
- How many tool calls were avoided?
- How many tokens were saved?
- Did it reduce hallucinations?
- Did it reduce latency?
- Did it improve completion accuracy?

## 8 Comparison

### 8.1 Without TMP

An agent often performs exploratory work before it can act:

```
read files -> search scripts -> inspect docs -> run help -> infer command ->
execute -> recover
```

Problems:

- Many tool calls.
- High token usage.
- Slow.
- Fragile.
- Easy to hallucinate flags.
- Hard to know if result is grounded.

### 8.2 With TMP

The agent asks for a mapped result:

```
compile map -> resolve intent -> invoke operation
```

Benefits:

- Fewer tool calls.
- Lower token use.
- Explicit failure when no map exists.
- Dynamic completions from resolvers.
- Reusable maps.
- Better audit trail.

### 8.3 MCP and Tool Schemas Compared

Model Context Protocol and tool schemas expose tools to agents. TMP is complementary.

MCP can define a tool such as "run command" or "query database". TMP can define the valid operations inside that tool and how to fill their parameters from local context.

In short:

- MCP exposes tools to a model.
- TMP maps valid operations behind tools.
- OpenAPI describes HTTP APIs.
- TMP can map API operations into agent and terminal workflows.
- Shell completion offers candidates.
- TMP can provide context-aware candidates from operation maps.

TMP should not compete with these systems. It should become the mapping layer they can use.

## 9 Product Goals

### 9.1 Goal 1: Build the CLI Adapter

The current `tmp` CLI is the first adapter. It should continue to support:

- Schema storage.
- Help-based draft generation.
- Registry install.

- Context compilation.
- Intent resolution.
- Command execution.
- Terminal-readable outputs.

## 9.2 Goal 2: Build the Completion Adapter

TMP should expose completion data for terminals:

- Operation completions.
- Parameter completions.
- Dynamic resolver values.
- Context-aware suggestions.

This is a strong early user-facing use case because it helps humans even before full agent integration.

## 9.3 Goal 3: Build the Agent Adapter

TMP should expose a stable agent-facing tool:

- `compile_context`
- `list_operations`
- `resolve_intent`
- `resolve_parameters`
- `invoke_operation`
- `explain_operation`

This can be exposed through CLI, MCP, or another adapter.

## 9.4 Goal 4: Build Schema Generation Workflows

AI agents can help generate maps, but generation should be outside the deterministic resolver path.

Agent generation workflow:

1. Inspect raw surface.
2. Draft map.
3. Attach evidence.
4. Run verification.
5. Save draft or verified schema.
6. Publish to registry if approved.

This keeps model usage optional and user-controlled.

## 9.5 Goal 5: Build the Registry

A registry lets teams share operation maps.

Registry entries should include:

- Surface type.

- Operation count.
- Verification level.
- Publisher.
- Version.
- Checksum.
- Compatibility notes.
- Risk metadata.

### 9.6 Goal 6: Build SQL and API Adapters

SQL and API adapters make TMP more than CLI.

SQL adapter goals:

- Map allowed query templates.
- Resolve table and column names.
- Classify read/write risk.
- Prevent arbitrary destructive queries by default.

API adapter goals:

- Import OpenAPI specs.
- Map endpoints to operations.
- Resolve environment and auth context.
- Classify side effects.

### 9.7 Goal 7: Build Workflow and Script Adapters

Workflow/script adapters should map:

- Inputs.
- Outputs.
- Preconditions.
- Side effects.
- Approval requirements.
- Dry-run behavior.

## 10 Benchmarking the Hypothesis

TMP has strong claims. They should be tested.

The main hypotheses are:

1. TMP reduces tool-call count.
2. TMP reduces token usage.
3. TMP reduces hallucinated commands or invalid operations.
4. TMP reduces time-to-correct-action.
5. TMP improves terminal completion relevance.
6. TMP improves agent success rate on multi-surface tasks.

## 10.1 Benchmark Design

Use paired tasks. Each task is run in two modes:

- Baseline: agent or user works without TMP.
- TMP-assisted: agent or user can call TMP.

Keep the repository, task, agent model, and environment constant.

## 10.2 Metrics

Measure:

- Number of tool calls.
- Input tokens.
- Output tokens.
- Wall-clock time.
- Whether the final operation was correct.
- Whether an invalid command/API/query was attempted.
- Number of user clarifications.
- Number of failed attempts.
- Whether approval was requested for high-risk actions.

## 10.3 Task Categories

Benchmark tasks should cover:

- CLI command resolution.
- Terminal completion.
- API operation selection.
- SQL read query selection.
- SQL write prevention.
- Workflow invocation.
- Custom script argument selection.
- Missing schema handling.
- Registry installation and reuse.

## 10.4 Example Benchmark Task

Task:

```
Run the unit tests for the parser package.
```

Baseline expected behavior:

- Agent searches files.
- Agent reads package metadata.
- Agent infers command.
- Agent runs command.

TMP expected behavior:

- Agent calls `tmp resolve "run parser unit tests"`.
- TMP returns mapped command.
- Agent runs `tmp run`.

Measure whether tool calls and tokens decrease.

## 10.5 Completion Benchmark

For terminal completion, measure:

- Precision at 1.
- Precision at 5.
- Time to completion candidate.
- Whether invalid candidates appear.
- Whether dynamic values match current context.

Example:

```
cargo run --bin <TAB>
```

Expected TMP behavior:

- Return only current workspace binaries.

## 10.6 SQL Benchmark

Task:

```
Show recent failed jobs.
```

Baseline:

- Agent inspects schema and writes SQL.

TMP-assisted:

- Agent resolves `sql.recent_failed_jobs`.
- TMP fills parameters and classifies query as read-only.

Measure:

- SQL validity.
- Number of schema-inspection calls.
- Whether mutating SQL was avoided.

## 10.7 Agent Benchmark Output

Every benchmark run should record:

```
{
  "task_id": "cli.parser_tests",
  "mode": "tmp_assisted",
  "tool_calls": 2,
  "input_tokens": 1200,
  "output_tokens": 300,
```

```
  "wall_time_ms": 4200,
  "success": true,
  "invalid_operation_attempted": false,
  "user_clarifications": 0
}
```

This makes TMP improvements measurable instead of anecdotal.

# 11 Architecture Risks

TMP can fail if its architecture becomes too vague. To avoid that, it needs firm boundaries.

## 11.1 Risk: Everything Becomes a Tool

If every possible action is mapped without structure, TMP becomes another noisy registry.

Correction:

- Use clear surface types.
- Require operation identity.
- Require parameter definitions.
- Require effect metadata.

## 11.2 Risk: Draft Maps Are Treated as Verified

Agent-generated maps may look convincing while still being wrong.

Correction:

- Mark generated maps as draft.
- Attach evidence.
- Require verification before high-trust use.

## 11.3 Risk: The CLI Becomes the Whole Protocol

The current implementation can bias the design toward shell commands only.

Correction:

- Treat CLI as one adapter.
- Design generic operation maps.
- Add SQL/API/workflow/script examples early.

## 11.4 Risk: TMP Becomes an AI Provider Wrapper

Provider integration creates credential and trust problems.

Correction:

- Keep generation optional and external.
- Let user-selected agents generate maps.
- Keep deterministic resolution separate from model calls.

# 12 Roadmap

## 12.1 Phase 1: Ground the Current CLI Implementation

- Stabilize schema storage.
- Improve deterministic help parsing.
- Keep generated schemas unverified.
- Improve context compilation.
- Improve resolver accuracy.

## 12.2 Phase 2: Define the General Operation Map

- Add surface types.
- Add effect metadata.
- Add evidence metadata.
- Add operation IDs.
- Add adapter boundaries.

## 12.3 Phase 3: Completion Prototype

- Expose completions for shell/terminal integration.
- Measure precision and latency.
- Support dynamic values.

## 12.4 Phase 4: Agent Adapter

- Add a small agent-facing interface for listing, resolving, and explaining operations.
- Support one-call intent resolution.
- Track tool-call and token reduction.

## 12.5 Phase 5: Registry

- Publish verified operation maps.
- Add checksums.
- Add compatibility metadata.
- Add trust levels.

## 12.6 Phase 6: SQL/API/Workflow Adapters

- Add OpenAPI import.
- Add SQL template maps.
- Add workflow operation maps.
- Add script operation maps.

## 12.7 Phase 7: Benchmark Harness

- Create benchmark scenarios.
- Record tokens, tool calls, latency, and success.
- Compare baseline against TMP-assisted flows.
- Publish results.

# 13 Conclusion

TMP should be framed as a protocol for mapping intent to verified operations across many executable surfaces. CLI commands are the first implementation, not the end state.

The important insight is that terminals, agents, workflows, APIs, SQL systems, and scripts all need the same thing: a grounded map from intent to valid action.

If TMP succeeds, it can become a shared infrastructure layer that improves:

- Terminal completion.
- AI agent reliability.
- Tool-call efficiency.
- Token efficiency.
- Command and query safety.
- Workflow discoverability.
- Registry-based reuse.

The next step is to ground the broad design through prototypes and benchmarks. TMP should not only claim fewer hallucinations and lower token usage. It should measure those claims and let the results guide the roadmap.